

Simple Submission Template

This starter is designed for students who need a working submission quickly and want code that is easy to read.

It is configured for the batch-distillation anomaly-detection competition.

At each timestep, the goal is to predict a binary anomaly label derived from:

Label (common/all)

Codabench evaluates submissions with **binary F1-score**.

Download

The starter stays deliberately small:

- no hidden framework
- no code upload to Codabench
- one plain `submission.csv` file for upload
- detailed comments in the Python files

Install

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Understand The Data Package

The downloadable `public_data` package contains:

- `train/`: 28 normal runs used for fitting a model
- `val/`: 10 labeled anomalous runs for local tuning
- `val_labels/`: labels for the validation runs
- `test/`: 53 unlabeled runs that require predictions for Codabench
- `manifest.json`: the source of truth for required numeric run IDs and row counts

The hidden leaderboard set is already bundled inside one `test/` folder. You submit predictions for that full folder once. Codabench computes the public and private scores internally from the same CSV.

Submission CSV Format

Upload one CSV file with exactly these columns:

```
run_id,timestep,prediction
1,0,0
1,1,0
1,2,1
```

Column meanings:

- `run_id`: integer test-run identifier from `manifest.json`
- `timestep`: zero-based row index inside that run
- `prediction`: binary anomaly prediction, `0` for normal or `1` for anomalous

Every timestep of every test run must appear exactly once.

The starter already includes `example_submission.csv`, which is a valid random submission in the exact upload format.

Run ID Correspondence Table

These numeric IDs are stable and intentionally arbitrary. They do **not** reveal which hidden runs belong to the public or private leaderboard split.

▼ Show the run_id to file mapping

run_id	released test file
1	test/series_001.csv
2	test/series_002.csv
3	test/series_003.csv
4	test/series_004.csv
5	test/series_005.csv
6	test/series_006.csv
7	test/series_007.csv
8	test/series_008.csv
9	test/series_009.csv
10	test/series_010.csv
11	test/series_011.csv
12	test/series_012.csv
13	test/series_013.csv
14	test/series_014.csv
15	test/series_015.csv
16	test/series_016.csv
17	test/series_017.csv

run_id	released test file
18	test/series_018.csv
19	test/series_019.csv
20	test/series_020.csv
21	test/series_021.csv
22	test/series_022.csv
23	test/series_023.csv
24	test/series_024.csv
25	test/series_025.csv
26	test/series_026.csv
27	test/series_027.csv
28	test/series_028.csv
29	test/series_029.csv
30	test/series_030.csv
31	test/series_031.csv
32	test/series_032.csv
33	test/series_033.csv
34	test/series_034.csv
35	test/series_035.csv
36	test/series_036.csv
37	test/series_037.csv
38	test/series_038.csv
39	test/series_039.csv
40	test/series_040.csv
41	test/series_041.csv
42	test/series_042.csv
43	test/series_043.csv
44	test/series_044.csv
45	test/series_045.csv
46	test/series_046.csv
47	test/series_047.csv
48	test/series_048.csv
49	test/series_049.csv
50	test/series_050.csv
51	test/series_051.csv
52	test/series_052.csv
53	test/series_053.csv

Quick Start

Create a valid random submission:

```
python predict.py --public-data /path/to/public_data --csv-path ./submission.csv
```

What this command does:

- reads the manifest to find all required test runs
- generates random binary predictions for every timestep
- writes one flat CSV that you can upload directly

If you only want to inspect the required format, open `example_submission.csv`.

Recommended Workflow

1. Use `train/` to fit your method.
2. Use `val/` and `val_labels/` to debug thresholds and compare ideas locally.
3. Generate predictions only for `test/`.
4. Upload the resulting `submission.csv` to Codabench.

Important: validation files are only for local experimentation. They must not be included in the uploaded CSV. The provided scripts already handle this correctly.

Inspect The Package First

```
python prepare_data.py --public-data /path/to/public_data
```

This prints a short JSON summary of the training, validation, and test splits.

Validate Your CSV

If you create a CSV yourself, validate it before uploading:

```
python make_submission.py --public-data /path/to/public_data --predictions-csv ./my_predictions.csv --csv-path ./submission.csv
```

This checks required columns, run IDs, row counts, timestep coverage, duplicate rows, and binary prediction values.

Optional PyTorch Baseline

Train the included autoencoder on the normal training rows:

```
python train.py --public-data /path/to/public_data --model-dir ./artifacts/model
```

Then generate a submission CSV in model mode:

```
python predict.py --public-data /path/to/public_data --strategy model --model-dir ./artifacts/model --csv-path ./submission.csv
```

Files

- `prepare_data.py`: loads manifests and aligned train, validation, and test runs
- `example_submission.csv`: valid random submission you can inspect as a format reference
- `model.py`: small feed-forward autoencoder
- `train.py`: fits the optional autoencoder baseline and saves `detector.pt`
- `predict.py`: one-command entry point that writes a submission CSV using random predictions or the model
- `make_submission.py`: validates and canonicalizes an existing submission CSV

Notes

- Random predictions are deterministic for a fixed `--seed`.
- `train.py` also uses a fixed seed by default (`--seed 13`) so repeated runs on the same machine are reproducible.
- The starter code is intentionally verbose in comments so the data flow is easy to follow.
- The optional model trains on normal rows only.
- The optional model threshold is chosen from the training reconstruction-error quantile.
- Predictions are binary `0/1`, not probabilities.